JAIN
DEEMED-TO-BE UNIVERSITY

JAIN
(DEEMED-TO-BE UNIVERSITY)
BENGALURU

AWS CLOUD & DATA ENGINEERING

PRACTICAL ASSIGNMENT REPORT



Submitted by

Name	:	**Chanikya Gunji**
Roll Number	:	23BTRCA055
College Name	:	Jain (Deemed-to-be University), Bengaluru
Batch	:	2027
Date	:	11 October 2026

Assignment Title

AWS CLOUD & DATA ENGINEERING

Technologies Used

SQL | Linux | Amazon S3 | AWS IAM | AWS Glue | AWS Glue Data Catalog | AWS Glue Visual ETL | Apache Parquet

Objective

To practice SQL queries and Linux commands, and to build an AWS data engineering pipeline that reads employee data from Amazon S3, discovers its schema using AWS Glue Crawler, transforms the data using AWS Glue Visual ETL, and stores the curated output in Amazon S3 in Parquet format.

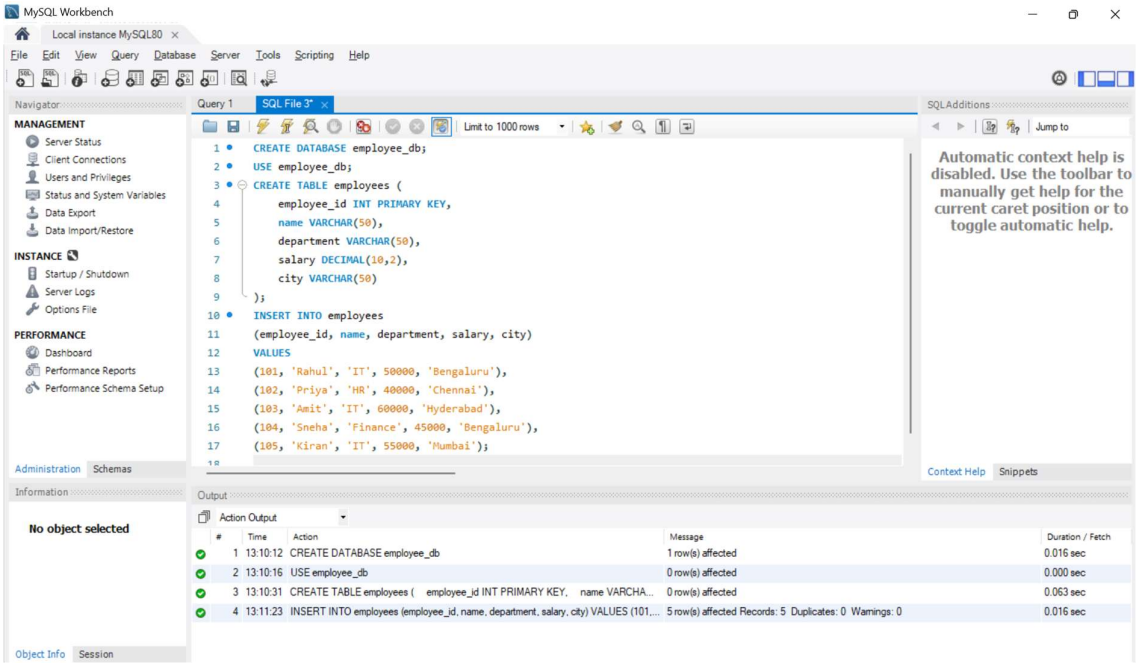


SQL Practice:

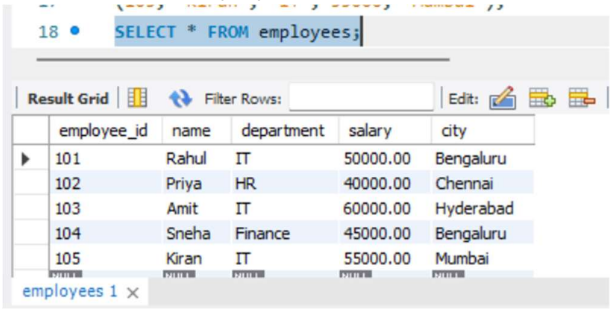
Create a database

create the employee table

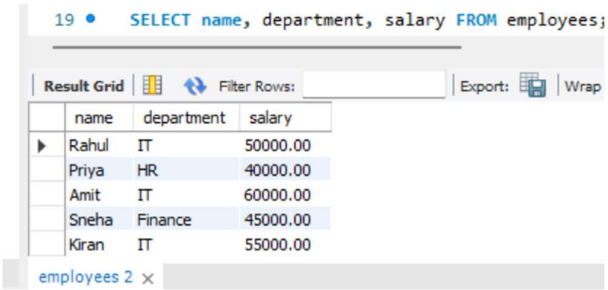
Insert the sample records



SELECT * FROM employees;



SELECT name, department, salary
FROM employees;



SELECT * FROM employees
WHERE department = 'IT';

```
20 •  SELECT * FROM employees WHERE department = 'IT';
```

Result Grid | Filter Rows: | Edit: | Exp

employee_id	name	department	salary	city
101	Rahul	IT	50000.00	Bengaluru
103	Amit	IT	60000.00	Hyderabad
105	Kiran	IT	55000.00	Mumbai
NULL	NULL	NULL	NULL	NULL

employees 3 ×

SELECT name, salary
FROM employees
WHERE salary > 50000;

```
21 •  SELECT name, salary FROM employees WHERE salary > 50000;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	salary
Amit	60000.00
Kiran	55000.00

SELECT name, salary FROM employees ORDER BY salary DESC;

```
22 •  SELECT name, salary FROM employees ORDER BY salary DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	salary
Amit	60000.00
Kiran	55000.00
Rahul	50000.00
Sneha	45000.00
Priya	40000.00

employees 5 ×

SELECT name, salary FROM employees ORDER BY salary ASC;

```
23 •  SELECT name, salary FROM employees ORDER BY salary ASC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	salary
Priya	40000.0
Sneha	45000.00
Rahul	50000.00
Kiran	55000.00
Amit	60000.00

Refresh data re-executing the original query

employees 6 ×

SELECT COUNT(*) AS total_employees FROM employees;

```
24 •  SELECT COUNT(*) AS total_employees FROM employees;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conte

total_employees
5

Result 7 ×

SELECT AVG(salary) AS average_salary FROM employees;

```
25 •  SELECT AVG(salary) AS average_salary FROM employees;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

average_salary
50000.000000

Result 8 ×

SELECT MAX(salary) AS highest_salary FROM employees;

```
26 •  SELECT MAX(salary) AS highest_salary FROM employees;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conte

highest_salary
60000.00

Result 9 ×

SELECT MIN(salary) AS lowest_salary FROM employees;

```
27 •  SELECT MIN(salary) AS lowest_salary FROM employees;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Co

lowest_salary
40000.00

SELECT department, COUNT(*) AS employee_count FROM employees GROUP BY department ORDER BY department;

```
28 • SELECT department, COUNT(*) AS employee_count FROM employees GROUP BY department ORDER BY department;
29 • SELECT name, city FROM employees WHERE city = 'Bengaluru';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

department	employee_count
Finance	1
HR	1
IT	3

Result 11 × Read Only

SELECT name, city FROM employees WHERE city = 'Bengaluru';

```
29 • SELECT name, city FROM employees WHERE city = 'Bengaluru';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	city
Rahul	Bengaluru
Sneha	Bengaluru

employees 12 ×

SELECT * FROM employees WHERE name LIKE 'A%';

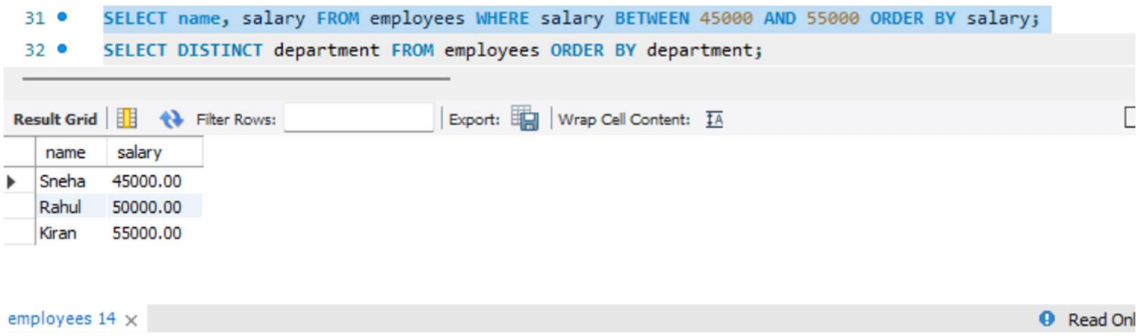
```
30 • SELECT * FROM employees WHERE name LIKE 'A%';
```

Result Grid | Filter Rows: | Edit:

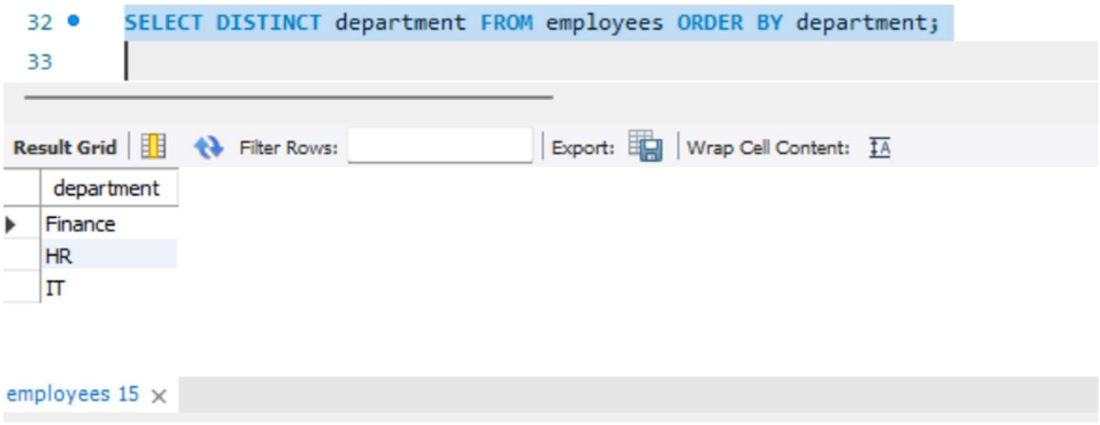
employee_id	name	department	salary	city
103	Amit	IT	60000.00	Hyderabad
NULL	NULL	NULL	NULL	NULL

employees 13 ×

SELECT name, salary FROM employees WHERE salary BETWEEN 45000 AND 55000 ORDER BY salary;



SELECT DISTINCT department FROM employees ORDER BY department;



Linux Practice in Ubuntu:

1. pwd — Show current directory
2. ls — List files
3. ls -l — Detailed file listing
4. mkdir aws_practice — Create folder
5. cd ~/aws_practice — Enter folder
6. touch employees.txt — Create file
7. echo "Employee data practice" > employees.txt — Write text
8. cat employees.txt — Display file contents

9. cp employees.txt employees_backup.txt — Copy file

10. mv employees_backup.txt backup.txt — Rename file

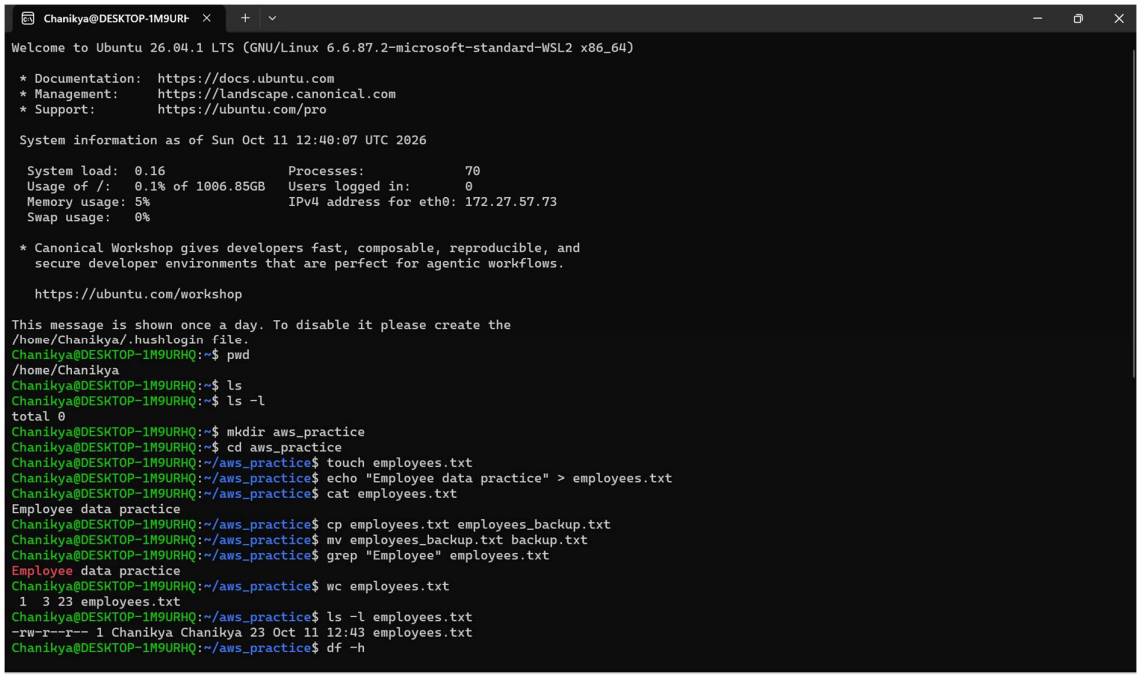
11. grep "Employee" employees.txt — Search text

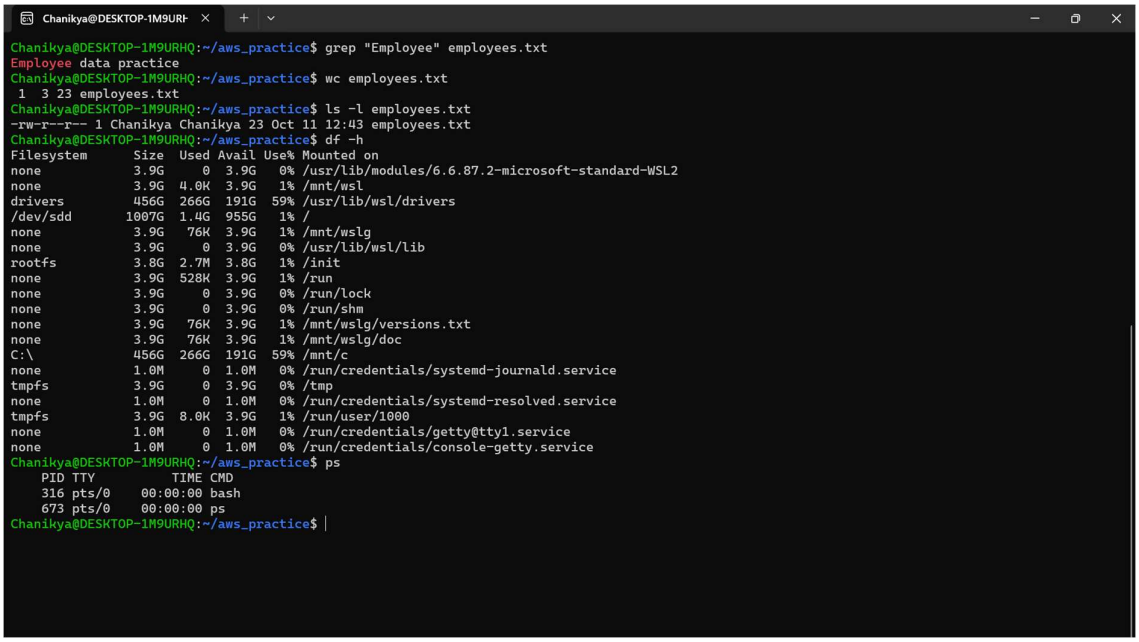
12. wc employees.txt — Count lines, words and bytes

13. ls -l employees.txt — Check file permissions

14. df -h — Check disk space

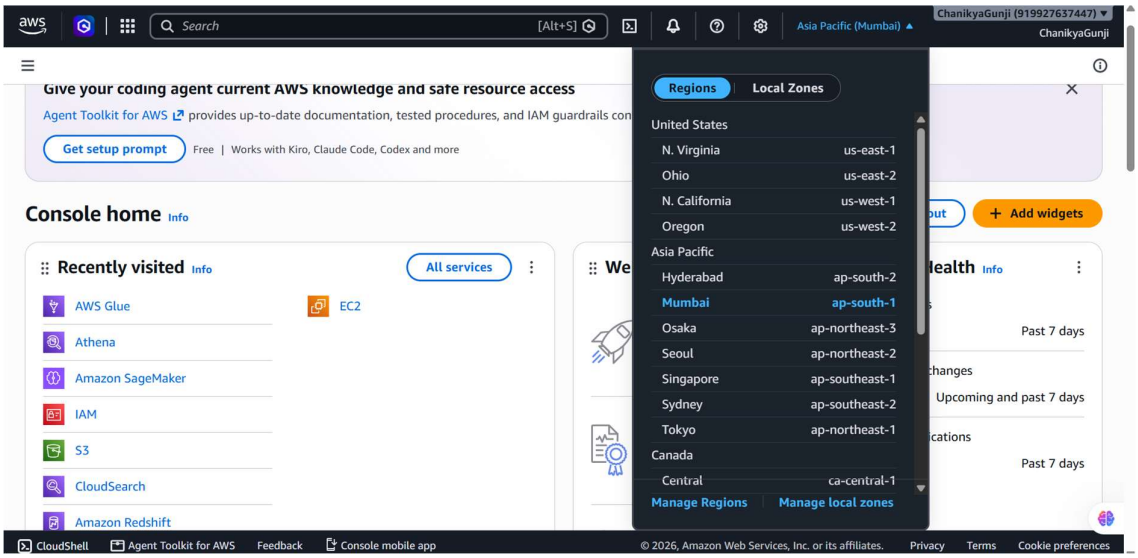
15. ps — View running processes



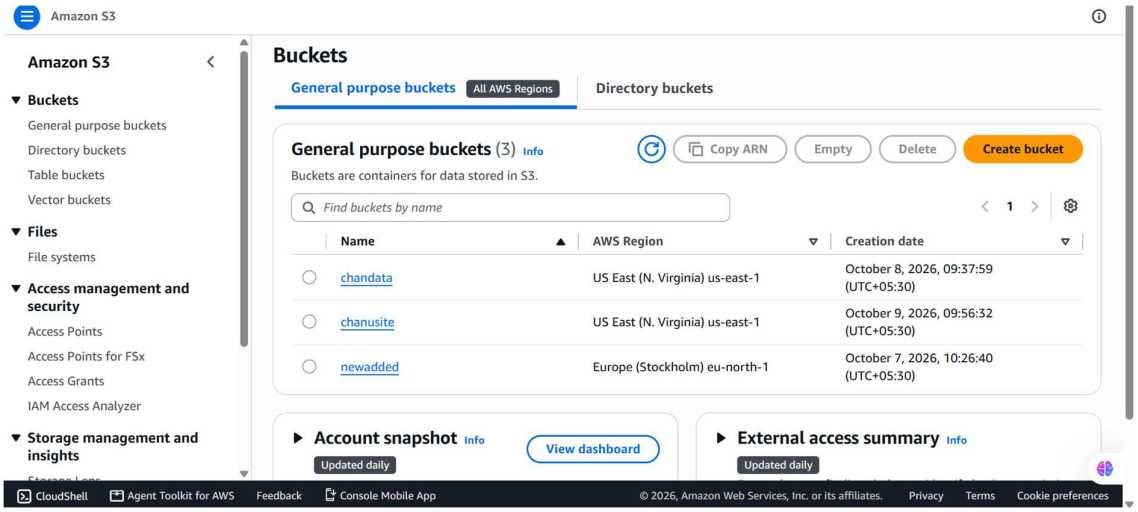


AWS Account / Region:

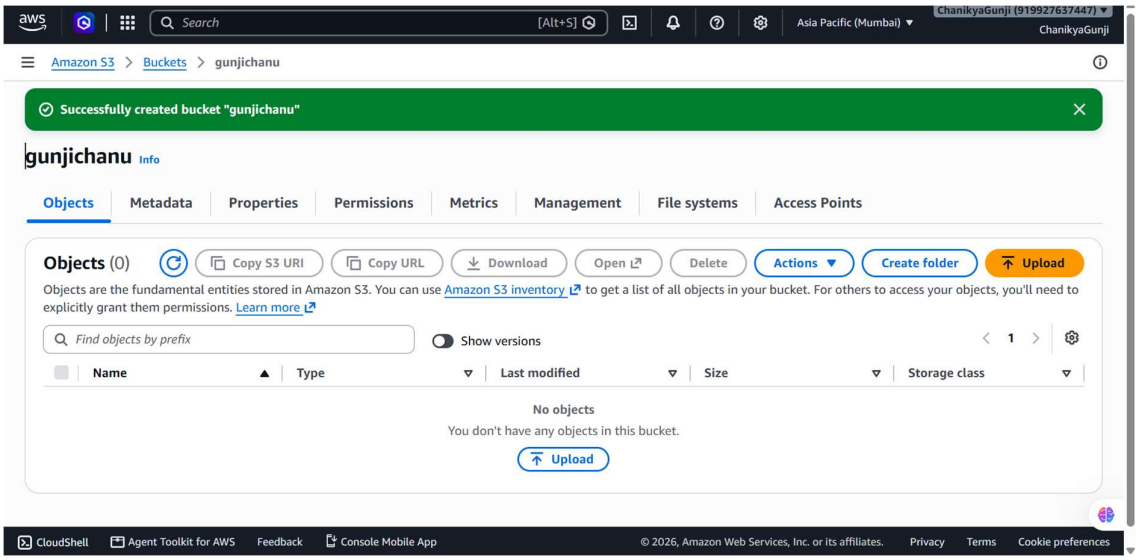
I accessed the AWS Management Console using my AWS account. I selected the AWS Region in which I am performing the cloud and data engineering tasks. All related AWS resources, including Amazon S3 and AWS Glue, will be configured consistently with the selected Region where required. A screenshot of the AWS Console and selected Region is included as evidence.



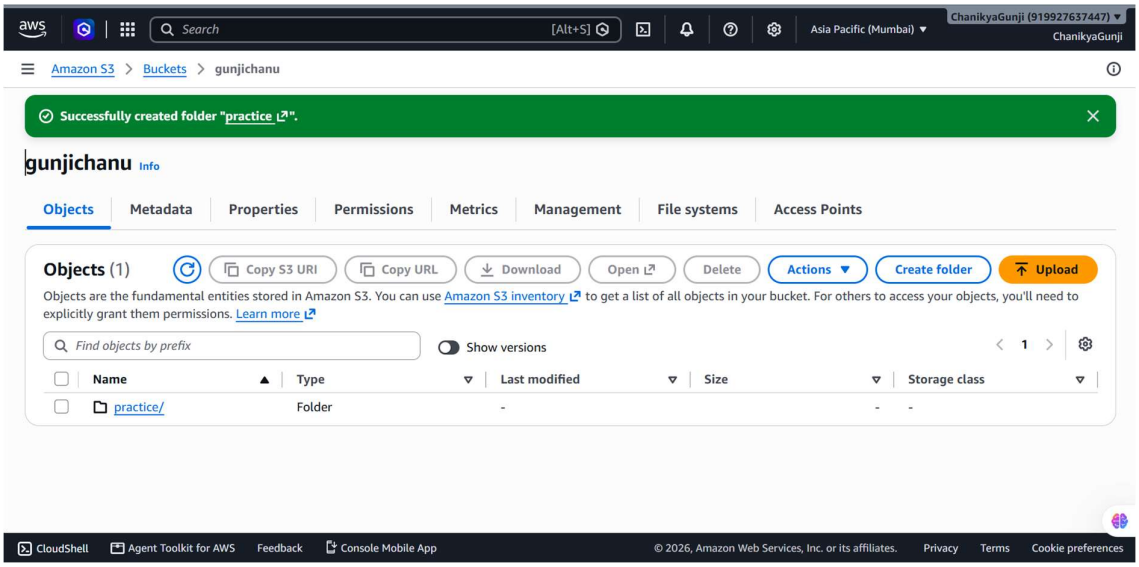
S3 bucket:



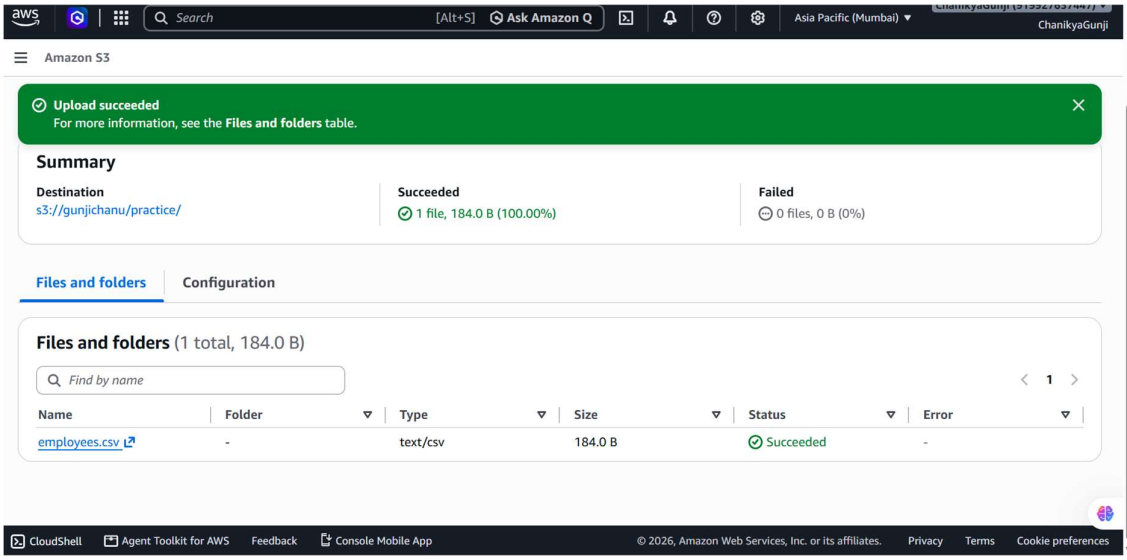
Created:



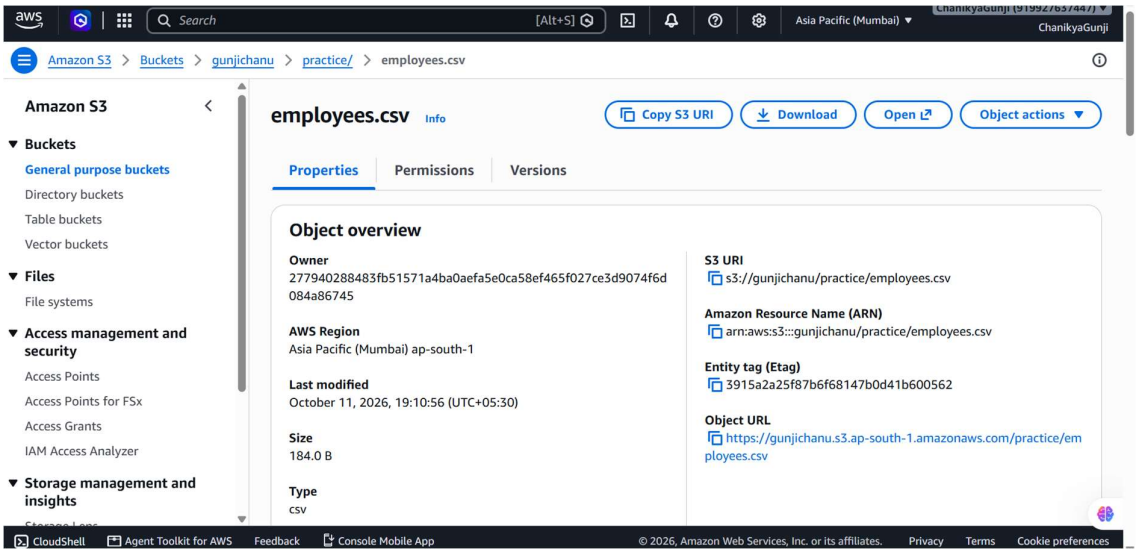
Folder:



employees.csv uploaded inside practice/:



Bucket name and file name should be visible:



IAM:

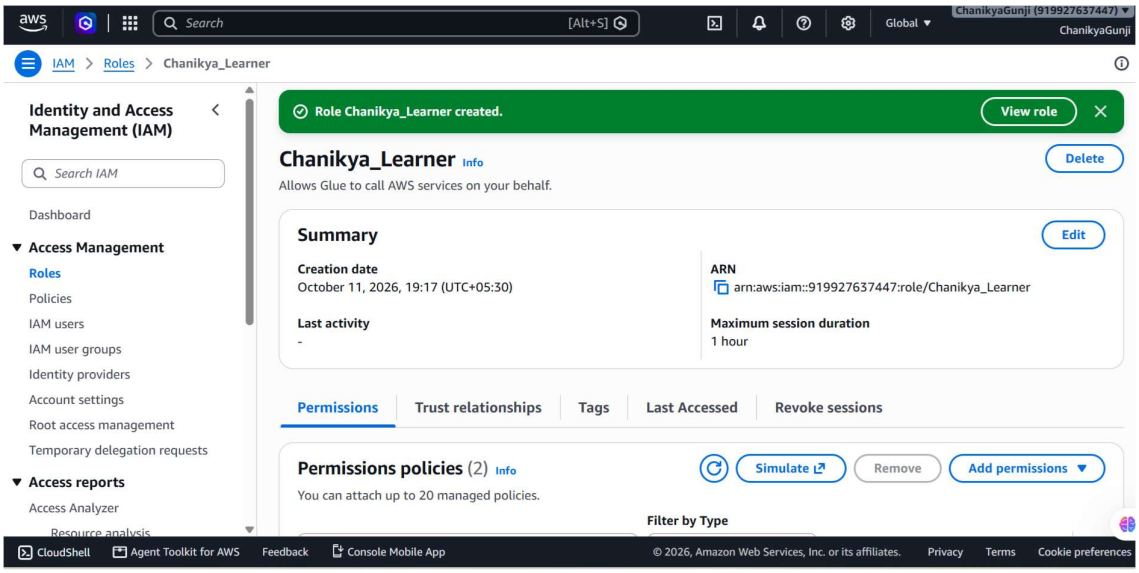
IAM Role used for AWS Glue

• Role name:Chanikya_Learner

Configure the trusted entity

On the Create role page:

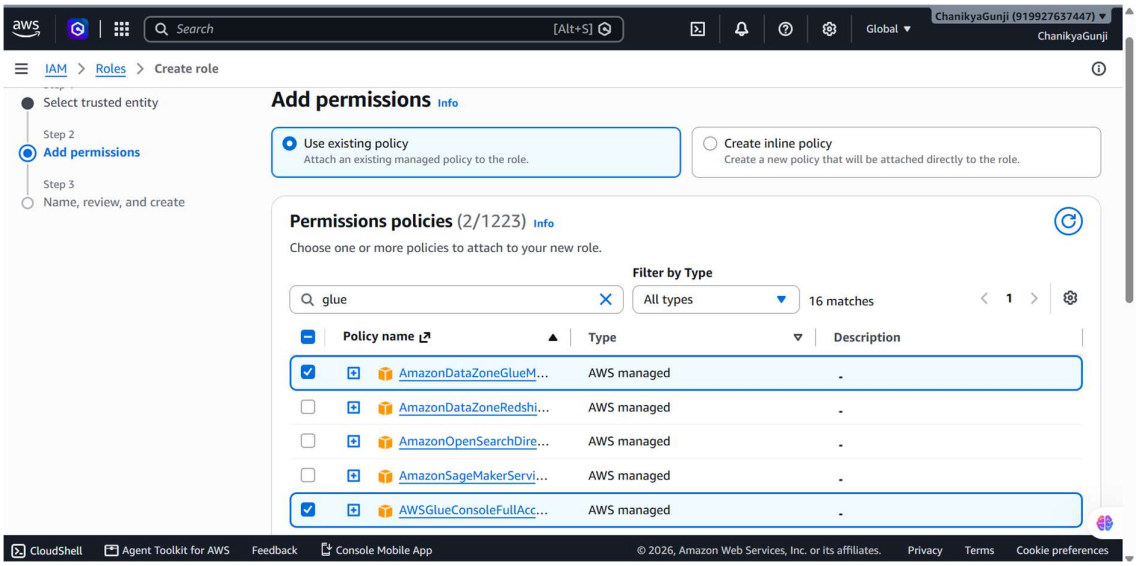
1. Select AWS service as the trusted entity type.
2. Under service or use case, select Glue.
3. Choose the Glue use case if prompted.
4. Click Next.



Attach permissions

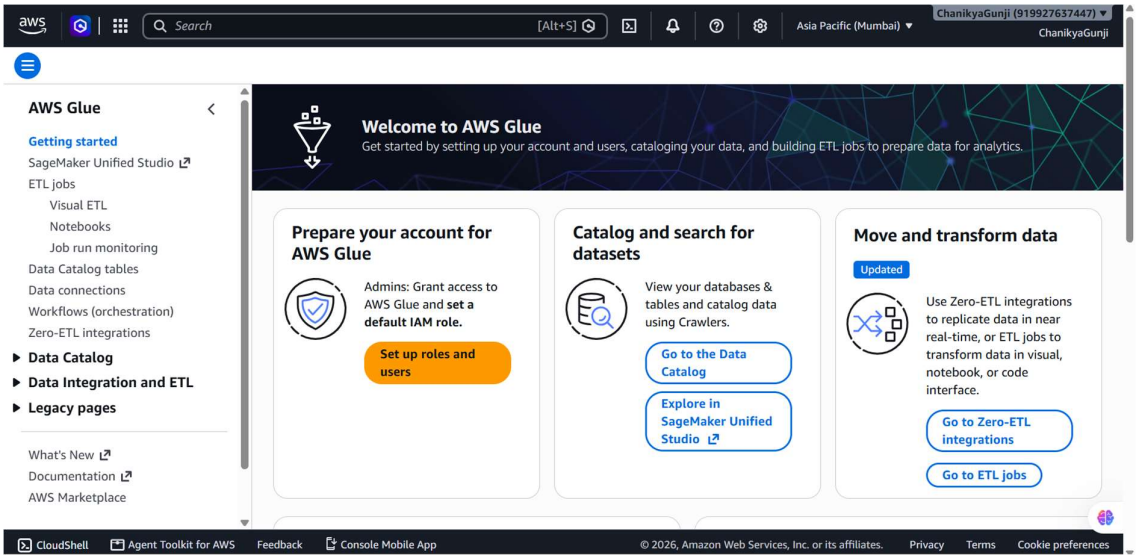
Search for and select:

AWSGlueServiceRole



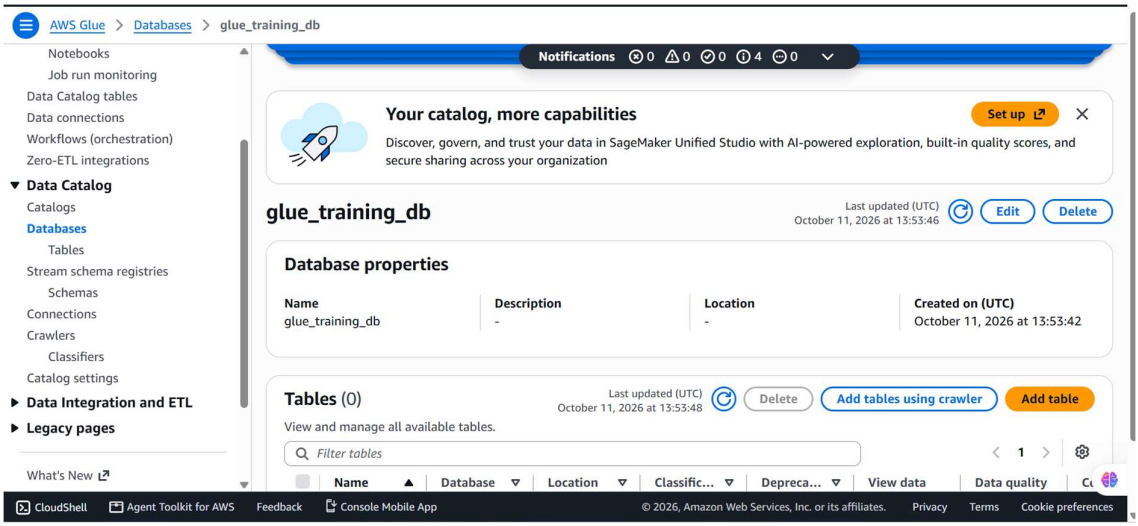
AWS Glue Database:

AWS Glue Console



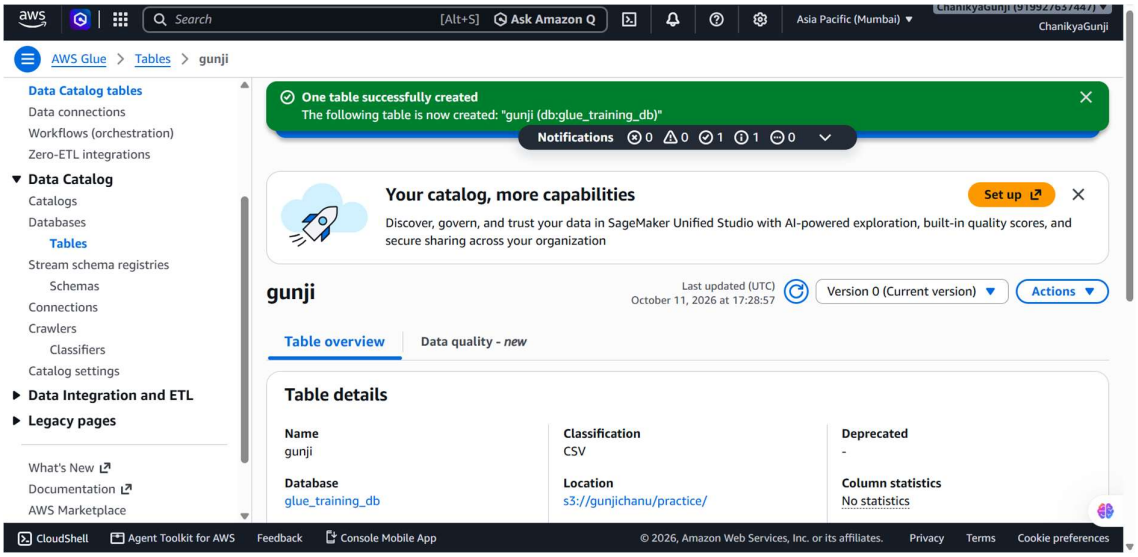
Data Catalog:

Database create

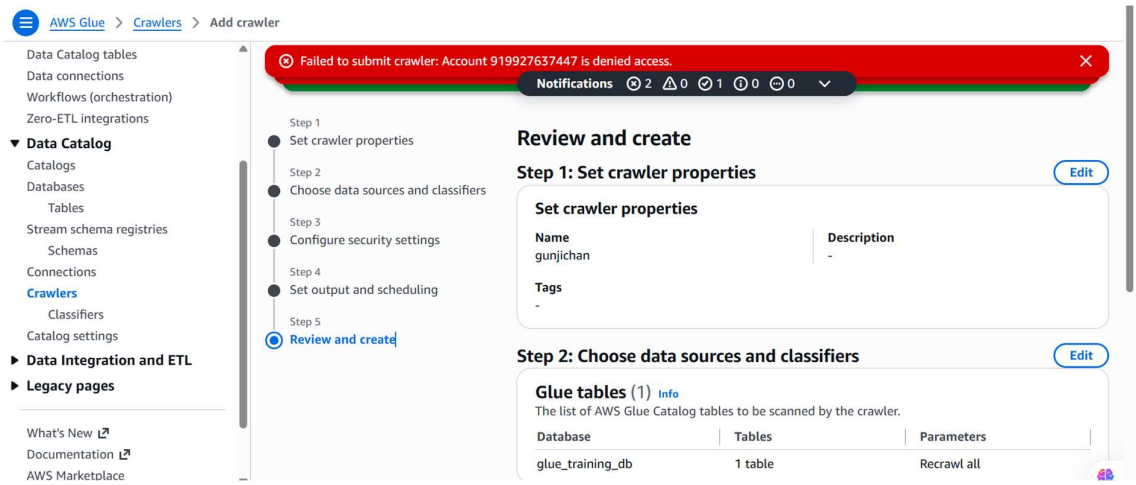


Glue Crawler:

Create Table



Crawler:



Upgrade Required

Crawler Execution:

Glue Data Catalog

- Database
- employees table
- Table successfully created

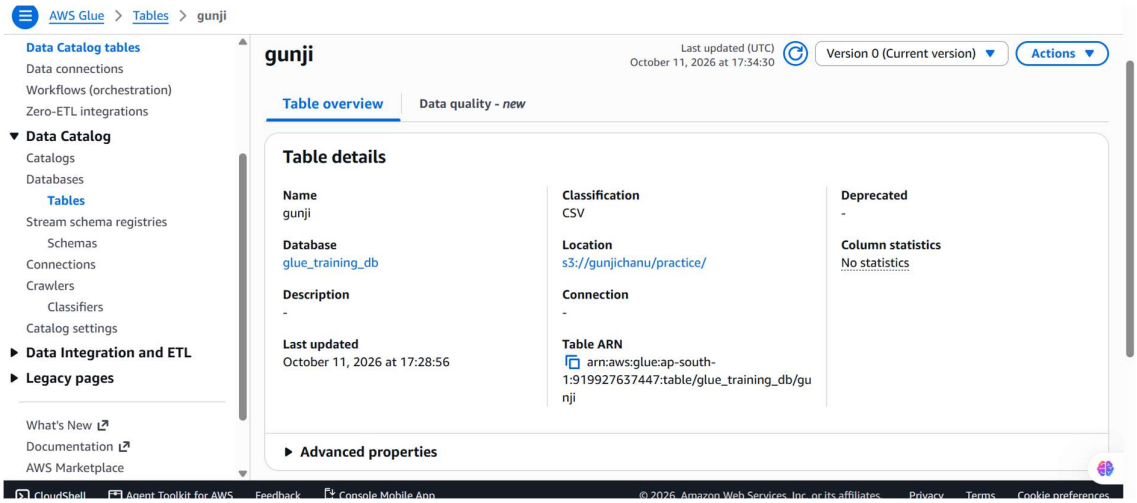
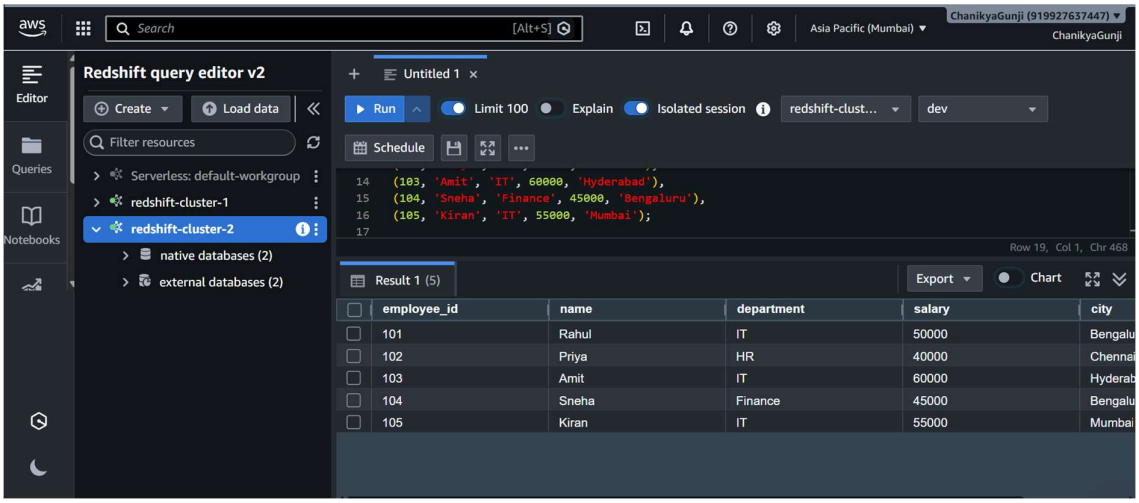


Table Schema:

• employee_id • name • department • salary • city • Data types should also be visible if possible.



Final Explanation

• Explain the complete workflow in your own words.

• S3 → Glue Crawler → Glue Data Catalog → Glue Visual ETL → Transformation → S3 Curated Data

Write 5–10 lines explaining what happens at each stage.

1. Amazon S3 — Raw Data

Amazon S3 is used to store the original employee data file. In this project, the file is named employees.csv and contains employee ID, name, department, salary and city. We upload it to the raw/ folder in our S3 bucket. This is the original data before any processing. Keeping raw data allows us to preserve the source for future use.

2. AWS Glue Crawler

The AWS Glue Crawler scans the CSV file stored in the S3 raw folder. It identifies the column names and data types, such as integer and string. It uses this information to determine the table structure. The crawler then creates or updates a table definition in the Glue Data Catalog. It helps us avoid defining the schema manually.

3. AWS Glue Data Catalog

The Glue Data Catalog is a central metadata repository. It stores information about the data, including the table name, column names, data types and S3 location. It does not normally store the actual employee records. Glue ETL jobs and other analytics services can use this metadata to understand and access the source data.

4. AWS Glue Visual ETL

Glue Visual ETL allows us to design a data processing pipeline using a visual editor. We add a source node to read the employee data and connect it to transformation nodes. We then configure a target node for the output. The visual editor helps us build the pipeline with less manual coding. When we run the job, AWS Glue processes the data according to the configured steps.

5. Data Transformation

During transformation, we clean and prepare the employee data for analysis. For example, we can filter employees whose salaries are greater than ₹45,000, select specific columns, remove unwanted records and convert data types. We can also standardize column names and handle missing values. These operations make the data more consistent and useful.

6. Amazon S3 — Curated Data

After transformation, AWS Glue writes the processed employee data to the curated/ folder in Amazon S3. This output is called curated data because it has been cleaned or transformed for further use. We can store it in CSV or Parquet format, depending on the project requirements. The curated data can later be queried using Amazon Athena or loaded into a data warehouse such as Amazon Redshift.

Final conclusion

This workflow demonstrates a basic ETL (Extract, Transform, Load) process. Amazon S3 stores the raw and processed data, AWS Glue Crawler discovers the schema, the Glue Data Catalog stores metadata, and Glue Visual ETL transforms the data. Finally, the processed records are saved in the curated S3 folder, making them ready for analytics and reporting.

Expected result: A raw employee CSV file is converted into a cleaned, structured dataset stored in S3 for downstream analysis.